

Unity Catalog Coding Assessment: Retail Sales Data Governance

Step 0 – Prerequisites

Requirements:

- Unity Catalog enabled in Azure Databricks
- Assigned metastore
- Admin access to create catalogs/schemas/tables
- Databricks SQL Warehouse or cluster with UC access
- Optional groups for testing:
 - analyst_group
 - pii_readers
 - west_region
 - east_region

Step 1 – Create Catalog and Schema

Code:

-- Create catalog for the demo

```
CREATE CATALOG IF NOT EXISTS retail_data  
COMMENT 'Retail Sales Data Catalog for Governance Demo';
```

-- Switch to the catalog

```
USE CATALOG retail_data;
```

-- Create schema for storing sales-related data

```
CREATE SCHEMA IF NOT EXISTS sales_data  
COMMENT 'Schema for storing sales transactions and customer details';
```

-- Use the schema

```
USE retail_data.sales_data;
```

Output:

```
OK
OK
OK
```

Step 2 – Create Sample Tables with Data

Code:

-- Customers table

```
CREATE OR REPLACE TABLE retail_data.sales_data.customers (  
  customer_id INT,  
  name      STRING,  
  email     STRING,  
  phone     STRING,  
  region    STRING  
) USING DELTA;
```

```
INSERT OVERWRITE retail_data.sales_data.customers VALUES  
(1, 'Asha Kumar', 'asha.kumar@example.com', '+91-90000-00001', 'west'),  
(2, 'Ravi Patel', 'ravi.patel@example.com', '+91-90000-00002', 'east'),  
(3, 'Neha Singh', 'neha.singh@example.com', '+91-90000-00003', 'west'),  
(4, 'Arjun Mehta', 'arjun.mehta@example.com', '+91-90000-00004', 'east');
```

-- Transactions table

```
CREATE OR REPLACE TABLE retail_data.sales_data.transactions (  

```

```
transaction_id INT,  
customer_id INT,  
transaction_date DATE,  
amount DECIMAL(10,2),  
payment_method STRING  
) USING DELTA;
```

```
INSERT OVERWRITE retail_data.sales_data.transactions VALUES  
(101, 1, DATE'2025-06-01', 1200.00, 'Card'),  
(102, 1, DATE'2025-06-15', 450.00, 'UPI'),  
(103, 2, DATE'2025-06-20', 999.99, 'Card'),  
(104, 3, DATE'2025-07-03', 150.00, 'Cash'),  
(105, 4, DATE'2025-07-07', 200.00, 'UPI'),  
(106, 3, DATE'2025-07-15', 750.00, 'Card');
```

Output:

```
OK  
4 rows affected  
OK  
6 rows affected
```

Step 3 – Data Discovery (Metadata & Tags)

Code:

-- Add comments and tags

```
COMMENT ON TABLE retail_data.sales_data.transactions IS 'Retail transactions  
(demo)';
```

```
COMMENT ON TABLE retail_data.sales_data.customers IS 'Customer master with PII-  
like columns (demo)';
```

```
ALTER TABLE retail_data.sales_data.customers
SET TAGS ('PII' = 'true', 'owner' = 'data-eng');
```

```
ALTER TABLE retail_data.sales_data.transactions
SET TAGS ('domain' = 'retail', 'gold' = 'false');
```

-- Query information_schema

```
SELECT table_catalog, table_schema, table_name, table_type, comment
FROM retail_data.information_schema.tables
WHERE table_schema = 'sales_data';
```

-- Show table properties

```
SHOW TBLPROPERTIES retail_data.sales_data.customers;
```

Output:

table_catalog	table_schema	table_name	table_type	comment
retail_data	sales_data	customers	BASE TABLE	Customer master with PII-like colum (demo)
retail_data	sales_data	transactions	BASE TABLE	Retail transactions (demo)
key		value		
PII		true		
owner		data-eng		

Step 4 – Data Access Control (GRANT/REVOKE)

Code:

-- Grant SELECT on transactions

```
GRANT SELECT ON TABLE retail_data.sales_data.transactions TO `analyst_group`;
```

-- Revoke SELECT on customers

```
REVOKE SELECT ON TABLE retail_data.sales_data.customers FROM `analyst_group`;
```

Step 5 – Dynamic Data Masking

Code:

```
CREATE OR REPLACE VIEW retail_data.sales_data.customers_masked AS
```

```
SELECT
```

```
customer_id,
```

```
name,
```

```
CASE
```

```
  WHEN is_member('pii_readers') THEN email
```

```
  ELSE concat(substr(email,1,3), '***@***.***')
```

```
END AS email_masked,
```

```
CASE
```

```
  WHEN is_member('pii_readers') THEN phone
```

```
  ELSE concat('***-****-', substr(phone, -2))
```

```
END AS phone_masked,
```

```
region
```

```
FROM retail_data.sales_data.customers;
```

Output:

customer_id	name	email_masked	phone_masked	region
1	Asha Kumar	ash***@.	+***-01	west
2	Ravi Patel	rav***@.	+***-02	east

Step 6 – Row-Level Security**Code:**

```
CREATE OR REPLACE VIEW retail_data.sales_data.transactions_rls AS
```

```
SELECT t.*
```

```
FROM retail_data.sales_data.transactions t
```

```
LEFT JOIN retail_data.sales_data.customers c
```

```
ON t.customer_id = c.customer_id
```

```
WHERE
```

```
(is_member('west_region') AND c.region = 'west')
```

```
OR (is_member('east_region') AND c.region = 'east')
```

```
OR (is_member('account_admin'));
```

Output:

transaction_id	customer_id	transaction_date	amount	payment_method
101	1	2025-06-01	1200.00	Card
102	1	2025-06-15	450.00	UPI
104	3	2025-07-03	150.00	Cash
106	3	2025-07-15	750.00	Card

Step 7 – Data Lineage**Code:**

```
-- Derived table 1
```

```

CREATE OR REPLACE TABLE retail_data.sales_data.monthly_sales AS
SELECT customer_id,
       date_format(transaction_date, 'yyyy-MM') AS month,
       SUM(amount) AS total_sales
FROM retail_data.sales_data.transactions
GROUP BY customer_id, date_format(transaction_date, 'yyyy-MM');

```

-- Derived table 2

```

CREATE OR REPLACE TABLE retail_data.sales_data.customer_monthly_sales AS
SELECT c.customer_id, c.name, c.region,
       m.month, m.total_sales
FROM retail_data.sales_data.monthly_sales m
JOIN retail_data.sales_data.customers c
ON m.customer_id = c.customer_id;

```

Output:

customer_id	month	total_sales
1	2025-06	1650.00
2	2025-06	999.99
3	2025-07	900.00
4	2025-07	200.00

Step 8 – Data Audit

Code:

```

SELECT event_time,
       user_identity.email AS user_email,
       action_name,

```

```
request_params.full_name AS object_full_name,  
response.result  
FROM system.access.audit  
WHERE request_params.full_name LIKE 'retail_data.sales_data.%%'  
AND action_name IN ('createTable','createView','select','grant','revoke')  
ORDER BY event_time DESC  
LIMIT 20;
```

Output:

event_time	user_email	action_name	object_full_name	result
2025-08-14T09:30Z	you@company.com	select	retail_data.sales_data.transactions	SUCCESS
2025-08-14T09:28Z	you@company.com	createTable	retail_data.sales_data.customer_monthly_sales	SUCCESS